情報科学演習 D

課題 1 レポート

【担当教員】　桝井 晃基，Olivier Nourry　教員

【提出者】　向井 大貴　(09B23081)
ソフトウェア科学コース・3 年
u230984j@ecs.osaka-u.ac.jp
【提出日】　2025 年 10 月 17 日

1　システムの仕様

`Lexer.run(String, String)` に字句解析器を実装する．第一引数に与えられた pascal ファイルを読み込み, 第二引数に与えられたファイルに字句解析結果を出力する．字句解析に成功した場合は”OK”を, ファイルが存在しない場合は”File Not Found”という文字列を返す．独自仕様として以下を定義する．

- 識別子が仕様に従わない (記号を含むなど) 場合,”Invalid identifier: line X”というエラーメッセージを出力する．
- それ以外のエラーが発生した場合,”Error”という文字列を返す．また, スタックトレースを標準出力に出力する．

字句解析の対象となる言語の仕様及び字句解析結果のトークン名,ID 及び出力形式は指導書に従う．

2　課題達成の方針と設計

トークンの優先順位　字句解析器は, トークンを優先順位つきで定義し, 正規表現によるマッチングを行うことで実装する．

トークンには優先順位が存在し, 例えば,”<=”という文字列は, トークン”<”と”=”に分割することもできるが, トークン”<=”として認識されるべきである．そのため,”<=”を”<”や”=”よりも優先順位の高いトークンとして定義し, 正規表現によるマッチングを行う際に”<=”が優先的に認識されるようにする．具体的なトークンの優先順位を表 1 に示す．なお, 数値が小さいほど優先順位が高いことを示す．

表 1: トークンの優先順位

優先順位	トークン	説明
1	コメント	コメントを最初に除去するため
2	文字列定数	文字列内に他のトークンが含まれる可能性があるため
3	以下に含まれない予約語, 記号, 整数定数	競合しないため
3	<=, >=, <>, :=	<や>,=より優先するため
3	:=, ..	: や=や.. より優先するため
4	<, >, =, :, ..	上記記号と競合するため
5	識別子	予約語と競合する可能性があるため

予約語を含む識別子への対応　予約語を単にその文字列だけで認識してしまうと, 例えば”beginner”という識別子が”begin”という予約語と”ner”という識別子に分割されてトークン化されてしまう．そのため, 予約語はそれだけが独立して存在している場合のみ認識されるようにする必要がある．これは, 正規表現の b(単語境界) を用いることで実現できる．例えば,”begin”という予約語は”begin b”という正規表現で定義する．

3　実装

3.1　パッケージ構成

今後の課題も含め, ソースコードのパッケージ構成は以下のようにした．

```
src
├── main.java.enshud  # メインパッケージ
│   ├── casl          # CASL2 関連
│   ├── pascal        # Pascal 関連
│   ├── token         # トークン関連
│   ├── s1.lexer      # 課題 1 字句解析器
│   ├── s2.parser     # 課題 2 構文解析器
│   ├── s3.checker    # 課題 3 意味解析器
│   └── s4.compiler   # 課題 4 コンパイラー
└── test.java.enshud  # テストコード
```

基本的に, 課題固有のコードは s1, s2, s3, s4 パッケージに実装し, 複数課題で共有するコードは casl, pascal, token パッケージに実装する.

3.2 トークンの定義

トークンの定義は, 列挙型 `pascal.TokenDef` に実装した. 各トークンは, トークン名, ID, 優先度, 正規表現パターンを持つ. また, 優先度順の処理を容易にするため, 優先度でソートしたリストを静的フィールドとしてキャッシュしておく. このとき, 優先度が同じ場合は正規表現の長さで比較し, 長い方を優先するようにした. これは, 優先度の設定ミスが存在しても, より具体的な正規表現が優先されるようにするためである.

```
public enum TokenDef {
    COMMENT(-1, "SCOMMENT", 1, "\\{[^}]*\\}"),
    AND(0, "SAND", 3, "and\\b"),
    ...
    STRING(45, "SSTRING", 2, "'([^']|'')*('|\\n)");

    public final int id; // トークン ID
    public final String tokenName; // トークン名
    public final int priority; // 優先度（数値が小さいほど優先度が高い）
    public final String regex; // 正規表現

    public static final List<TokenDef> SortedTokens = Arrays.stream(values())
    .sorted((t1, t2) -> {
        if (t1.priority != t2.priority) {
            return Integer.compare(t1.priority, t2.priority);    // 優先度で比較
        } else {
            // 正規表現の長さで比較（長い方が優先）
            return Integer.compare(t2.regex.length(), t1.regex.length());
        }
    })
    .toList();

    // コンストラクタ
    TokenDef(int Id, String tokenName, int priority, String regex) {
        this.id = Id;
        this.tokenName = tokenName;
        this.priority = priority;
        this.regex = regex;
    }
}
```

3.3 トークンの分離

トークンの分離は, クラス `Tokenizer` に実装した. このクラスは, インスタンス生成時に入力文字列を受け取り, メソッド `nextToken()` で次のトークンを返す. `nextToken()` は, 現在の位置から文字列の末尾まで

をスキャンし, 優先順位順にトークンの正規表現とマッチングを試みる. マッチした場合, そのトークンが文頭に存在する場合はそのトークンと行数を, そうでない場合は null を返す. マッチした場合, 現在の位置をマッチしたトークンの末尾に更新する. なお, トークンの前に空白文字が存在する場合, その空白文字はスキップし, 現在地を更新する.

例外として, 注釈トークンを認識した場合は, そのトークンは捨てて `nextToken()` を再帰的に呼び出し, その結果を返す. また, どのトークンともマッチしなかった場合, 識別子に不正な文字が含まれているものとして扱う. この場合, `TokenizerException` をスローし, その旨を示すエラーメッセージと行数を格納する.

3.4 字句解析結果の読み込み・保存

字句解析結果の ts ファイルへの保存・読み込みは, クラス `token.TokenFile` に実装した. このクラスは, 静的メソッド `SaveTokens` で字句解析結果 (Token 型のリスト) を ts ファイルに保存し, メソッド `getTokens` で ts ファイルから字句解析結果を読み込む. ファイル形式に誤りがある場合, `FileFormatException` をスローする. 今回の課題では,ts ファイルを読み込むことはないが, 今後の課題で必要になるため実装した.

3.5 字句解析器の実装

字句解析器は, クラス `Lexer` の run メソッドに直接実装した. このメソッドは, 第一引数で与えられた pascal ファイルを読み込み, 字句解析を行い, 第二引数で与えられたファイルに字句解析結果を保存する. まず, 入力ファイルが存在するかを `Files.exists` で確認し, 存在しない場合は”File Not Found”を返す. 次に, 入力ファイルを `Files.readAllBytes` を用いて一括で読み込み, 文字列として保持する.

その後, この文字列を `Tokenizer` に渡し, トークンを一つずつ取得していく. この処理は,Tokenizer の `hasNext()` が false を返すまで繰り返し, `nextToken()` でトークンを取得する.

問題なくトークンの分離を行えた場合, 例外を発生させることなくループを抜けるため, 字句解析結果を ts ファイルに保存し,”OK”を返す. ts ファイルへの保存には, クラス `token.TokenFile` の静的メソッド `SaveTokens` を用いる.

4 考察・工夫点

テストケースの追加

用意されているテストケースのみでは, 字句解析器の正しさを十分に検証できないと考えたため, 独自にテストケースを追加した. 具体的には, 以下のようなケースを追加した.

- normal01: 予約語を含む識別子 (例: ”beginner”, ”ifelse”)
- lexerr01: 識別子に不正な文字が含まれるケース (例: ”hoge_hoge”, ”fuga-fuga”)

タスクランナーの導入

開発をするうえで, ローカルでのテストやビルド,Jenkins でのジョブ実行など複数の操作を行うことがある. これらの操作を簡単かつ統一的に行うために, タスクランナー (Task) を導入した. 本来は Gradle でも同様のことが可能であるが,build.gradle が編集禁止となっているため, 別途 Task を導入した. Task では, 設定ファイル (Taskfile.yml) にコマンド実行などのタスクを定義し,task コマンドで簡単に実行することができる.

例えば, ビルドは `task build` で,Jenkins での Lexer ジョブの実行は `task jenkins-s1` で行うことができる. なお,Jenkins ジョブの実行には, 事前に Jenkins の API トークンを発行しておき, これを用いて Jenkins API を叩くことで実現している.

テストの自動化

Jenkins でテストジョブを実行する操作を毎回行うのは手間であるため, リモートへのプッシュ時に自動でジョブを走らせるようにした. 本来,Jenkins 側で設定を行い,git への push をトリガーにジョブを走らせることも可能である. しかし, 今回はその権限がないため, ローカルで git のフックを用いて実装した. 具体的には, リモートへの push を行う直前にジョブを走らせるように, ローカルリポジトリの `.git/hooks/pre-push` にスクリプトを追加した.

このスクリプトでは, プッシュ時の差分を取得し, その差分に s1 ディレクトリ以下のファイルが含まれている場合にのみ,Jenkins での Lexer ジョブを実行するようにしている. これは s2, s3, s4 ディレクトリでも同様であり, これら以外のファイルが変更された場合はすべてのジョブを走らせるようにしている. ジョブの実行には, 前述の Task を用いている.

今後を見据えた実装

今回の課題では, 字句解析器のみを実装すればよいが, 今後の課題で構文解析器, 意味解析器, コンパイラーを実装する必要がある. そのため, 今後の課題で利用するようなクラスやメソッドは,s1 パッケージではなく共通パッケージに実装し, 他課題からも利用できるようにした. また,TokenDef は enum で実装しているため, インスタンスなどを気にすることなくそのまま比較することができる. そのため,Parser でトークンでの条件判定が容易に行えるようになっている.

DFA を用いた字句解析器の実装

実装に当たり,DFA を用いて字句解析を行うことも考えたが, バグを生む可能性が高くこの段階でバグを生むことは今後の開発に大きな影響を与えると考えたため正規表現を採用した. 今後, 課題 4 のコンパイラーの実装まで終えた後, 余裕があれば DFA を用いた字句解析器の実装に挑戦したいと考えている.

5 感想

以降の課題では, それ以前の課題で実装したコードを利用することになるため, 各課題ごとでのコードの完成度ではなく, 全体を通してのコードの完成度が重要になると感じた. 今回, できるだけ早くすべての実相を進めることを目標にしており, 実際課題 1 提出までに Checker までの実相を終えることができた. 実際にこの方法をとって, 全体の完成度を高めることにつながっていると感じたため, 今後もこの方法を継続していきたいと思った.